

Product Roadmap 2026

Strategic Themes, Initiatives & Technical Plans | Product Leadership Team

1. Product Vision and Strategic Context

TechViet's product strategy for 2026 is anchored in the vision of becoming the leading AI-powered customer engagement platform in Southeast Asia. The strategy is informed by three macro-trends: first, the rapid adoption of large language models in enterprise customer service (Gartner predicts that by 2027, 40% of customer service interactions in developed markets will be handled by AI, up from 15% in 2025); second, the growing demand for multilingual support across Southeast Asian markets (our customers increasingly serve customers in Vietnamese, English, Thai, Indonesian, and Japanese); and third, the tightening of data protection regulations across the region, creating a competitive advantage for vendors like TechViet that can offer on-premises or in-region data processing. The 2026 roadmap is organized into four strategic themes, each with specific initiatives and measurable outcomes. The themes were developed by the Product Leadership team (VP Product Tran Ngoc Anh, Director of Product Le Thanh Tam, and the four Product Managers) in consultation with the CEO and CTO, and validated through interviews with 15 enterprise customers and analysis of 450 feature requests from the product feedback portal. The four themes are: AI Excellence (making the AI smarter and more accurate), Platform Scalability (supporting larger enterprises with more complex requirements), Developer Experience (making it easier to integrate and customize ChatAssist), and Compliance and Trust (meeting evolving regulatory requirements and building customer confidence in AI-powered interactions).

2. Theme 1: AI Excellence

2.1 RAG 2.0: Next-Generation Knowledge Retrieval

The current Knowledge Base v2 (released December 2025) uses a hybrid BM25 + dense vector retrieval approach that achieves 78% retrieval precision on our benchmark dataset. While this is a significant improvement over v1, customer feedback indicates that the system still struggles with several common scenarios: multi-hop questions that require synthesizing information from multiple documents (e.g., 'What is the refund policy for orders placed during the Lunar New Year promotion?'), questions that require understanding tables and structured data embedded in documents, and domain-specific terminology that the general-purpose embedding model does not capture well. RAG 2.0 will address these gaps through three technical improvements: first, a multi-stage retrieval pipeline that performs an initial broad retrieval (top-50 results using BM25 + dense vectors), followed by a re-ranking stage using a cross-encoder model that considers the interaction between the query and each candidate passage, followed by a synthesis stage that combines information from the top-5 re-ranked passages into a coherent response. The cross-encoder re-ranking is expected to improve precision by 15-20% based on offline experiments. Second, a table-aware retrieval system that can parse tabular data in documents (both HTML tables and tables in PDFs/images), convert them to a structured format, and answer queries that require table lookups or aggregations. This feature is particularly important for our financial services customers who have extensive policy documents with complex fee schedules and comparison tables. Third, a domain adaptation framework that allows each customer to fine-tune the embedding model on their specific

domain vocabulary and terminology, using a few-shot learning approach that requires only 50-100 labeled examples (query-document pairs). The domain adaptation will run as a scheduled training job in the customer's dedicated namespace on the GPU cluster, with the fine-tuned model weights stored in the customer's isolated model registry. Timeline: Q1 2026 — cross-encoder re-ranking (backend implementation and A/B test), Q2 2026 — table-aware retrieval (depends on improved document parsing pipeline), Q3 2026 — domain adaptation framework (depends on GPU cluster expansion in the cloud migration). Success metrics: retrieval precision > 88% on the benchmark dataset, customer-reported 'answer not found' rate < 10% (currently 18%), and at least 5 enterprise customers using domain adaptation by end of Q3.

2.2 Agentic AI: Multi-Step Task Completion

Currently, ChatAssist's AI handles each user message independently — it retrieves relevant information, generates a response, and returns it in a single turn. For 2026, we are building an agentic AI capability that enables the AI to plan and execute multi-step tasks, maintaining context across turns and interacting with external systems through a set of configurable 'tools'. Example use case: a customer support AI agent for an e-commerce company receives the query 'I want to return the shoes I ordered last week and get a refund'. The agentic AI would: (1) look up the customer's recent orders by calling the order management API, (2) identify the shoe order and check the return eligibility based on the return policy knowledge base, (3) initiate the return process by calling the returns API, (4) calculate the refund amount based on the payment method and return shipping costs, and (5) confirm the return details with the customer and provide the return shipping label. Each step involves a different 'tool' (API call, knowledge base query, calculation), and the AI orchestrates the sequence based on the user's intent and the results of each step. The technical architecture for agentic AI uses a planner-executor model: the Planner component receives the user's message and the conversation history, identifies the user's intent, and generates a plan (sequence of steps) to fulfill the intent. The Executor component executes each step in the plan, calling the appropriate tool and feeding the result back to the Planner for the next step. The Planner can revise the plan dynamically based on intermediate results (e.g., if the return eligibility check fails, the Planner adjusts the plan to explain the reason to the customer instead of proceeding with the return). Safety is a critical concern for agentic AI, as the AI will be taking actions with real-world consequences (e.g., processing refunds, updating account information). We are implementing a multi-layer safety framework: tool-level permissions (each tool has a permission level: read-only, write with confirmation, write without confirmation), action-level guardrails (e.g., refund amount must not exceed the original order value, account deletion requires human approval), and conversation-level monitoring (a separate safety model monitors the conversation for signs of manipulation or adversarial behavior and can halt the agent if detected). Timeline: Q2 2026 — tool framework and planner architecture (internal alpha), Q3 2026 — beta release with limited tool set (knowledge base, FAQ, simple read-only API calls), Q4 2026 — GA release with write actions and safety framework. This is the highest-priority initiative for 2026 as it represents a step-change in the value ChatAssist can deliver to customers.

3. Theme 2: Platform Scalability

As TechViet moves upmarket toward larger enterprise customers, the platform must scale to support significantly higher volumes, more complex organizational structures, and stricter performance requirements. The scalability theme encompasses three initiatives: Initiative 3.1: Multi-Region Architecture. Currently, all ChatAssist infrastructure runs in a single data center (or, post-migration, a single AWS region). For enterprise customers with global operations, this creates latency issues for users outside Southeast Asia and raises data residency concerns for customers with European or Japanese operations. The multi-region initiative will deploy ChatAssist compute infrastructure in three regions (Vietnam, Singapore, and Frankfurt) with data routing rules that ensure user data stays in the designated region. The conversation data will be partitioned by organization, with each organization assigned to a

home region based on their data residency requirements. Cross-region conversations (e.g., a Vietnamese agent serving a European customer) will use a sync protocol that minimizes cross-region data transfer while maintaining real-time message delivery. Initiative 3.2: Enterprise Multi-Tenancy Hardening. Current multi-tenancy is implemented at the application layer (row-level security in PostgreSQL, namespace isolation in Kubernetes). For our largest enterprise customers (particularly those in regulated industries like banking and insurance), we need to offer stronger isolation guarantees: dedicated database instances, dedicated compute resources, network isolation, and independent scaling. The implementation uses Kubernetes virtual clusters (using the vCluster project) to provide each enterprise customer with a dedicated virtual Kubernetes cluster that appears as a full cluster to the customer's admin but shares the underlying physical infrastructure for cost efficiency. Initiative 3.3: 10x Scale Engineering. Internal load testing has identified several bottlenecks that would prevent the platform from scaling to 10x the current load (from 50,000 to 500,000 concurrent connections). The key bottlenecks are: the PostgreSQL primary database (write throughput limited to approximately 5,000 transactions per second), the Redis cluster (memory pressure at 500,000 concurrent sessions), and the WebSocket infrastructure (kernel-level connection limits on the current node configuration). The 10x engineering initiative will address these through: PostgreSQL sharding using Citus extension (partitioning by organization_id), Redis cluster expansion with automatic shard management, and migration to a dedicated WebSocket gateway service using Go's epoll-based connection handling that can support 100,000 connections per pod.

4. Theme 3: Developer Experience

The Developer Experience theme focuses on reducing the friction of integrating ChatAssist into customer applications and enabling deeper customization without requiring TechViet's Professional Services team. This theme directly addresses the feedback from customer interviews: 60% of enterprise customers cited 'integration complexity' as their top frustration, and 45% have custom development needs that they currently fulfill through expensive Professional Services engagements. Initiative 4.1: Low-Code Bot Builder. A visual drag-and-drop interface for designing conversation flows, configuring AI behavior, and connecting to external systems. The Bot Builder targets business users (e.g., customer service managers, marketing teams) who can design simple chatbot experiences without developer involvement. The technical implementation uses a React-based flow editor (built on the ReactFlow library) that generates a declarative JSON configuration consumed by the ChatAssist runtime. The flow supports conditional branching (if-then-else based on user input or external data), integration nodes (REST API calls to external systems), AI nodes (knowledge base search, generative response, classification), and human handoff nodes (escalation to a human agent with context transfer). Initiative 4.2: Plugin Marketplace. An ecosystem where TechViet, partners, and customers can publish reusable plugins that extend ChatAssist's functionality. Plugins are sandboxed WebAssembly modules that run in a secure runtime within the ChatAssist infrastructure. The plugin API provides hooks for: message pre-processing (e.g., language detection, sentiment analysis), response post-processing (e.g., formatting, content filtering), tool integration (adding new tools for the agentic AI to use), and analytics extensions (custom metrics and dashboards). The marketplace will launch with 10 first-party plugins (built by TechViet) and a partner program for third-party developers. Initiative 4.3: Comprehensive SDK and Documentation Overhaul. Expand the SDK offering from the current TypeScript and Python SDKs to include Java, Go, Ruby, and PHP SDKs, covering all API v3 endpoints. Rebuild the developer documentation site using a Docusaurus-based architecture with interactive API playground, code examples for every endpoint in all supported languages, and step-by-step tutorials for common integration scenarios. The documentation will include a dedicated 'quickstart' guide that enables a developer to send their first message via the API within 5 minutes.